CSA17 – Artificial Intelligence: Assignment 3 (CO3) – Answers

Note. Student A / Student B follow the roles given in the question paper. Figures marked *(measured)* come from code I ran for Tasks 1 and 2. All other numbers are derived analytically or are exact counts. Where the paper asks for a measurement I did not run, I give the derived result or the expected outcome instead and label it so.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns

Part A – Student A: design and mathematical analysis

Symbols. Each square [i,j] has P (pit), W (wumpus), B (breeze) and S (stench), so the count is **$m = 4n^2$**.

Sentences.

Group	Count
$\neg P_{11}$, $\neg W_{11}$	2
Breeze rule $B_{ij} \Leftrightarrow$ (pit in a neighbour), one per square	n^2
Stench rule $S_{ij} \Leftrightarrow$ (wumpus in a neighbour), one per square	n^2
At least one wumpus	1
At most one wumpus: $\neg W_a \vee \neg W_b$ for every pair	$C(n^2,2)$
Total (before percepts)	**$2n^2 + C(n^2,2) + 3$**

Models. There are 2^m = 2^($4n^2$) models.

n	symbols	sentences	models
2	16	17	65,536
3	36	57	6.9×10^{10}
4	64	155	1.8×10^{19}
5	100	353	1.3×10^{30}

Minimal symbol set for one query. To prove "[2,2] is safe" ($\neg P_{22} \wedge \neg W_{22}$) we need only the rules of the visited squares and their percepts.

- Pit side (9 symbols): B_{11}, B_{12}, B_{21}, P_{11}, P_{12}, P_{21}, P_{22}, P_{13}, P_{31}. That is $2^9 = 512$ models.
- Wumpus side: another 9 symbols and 512 models.
- The count is independent of n. Only the visited squares matter, so the cost of a query does not grow with the board.

Soundness and completeness of model checking.

- *Sound*: it answers "entailed" only if α is true in every model where KB is true. It checks every model, so it never returns a wrong "yes".

- *Complete*: it always terminates with a correct answer, because the set of models is finite and is fully enumerated. No entailed sentence is missed.
- The price is time of 2^m, and the checker needs a finite number of symbols.

Part A – Student B: experimental validation *(measured)*

Scenario for n = 3 and 4: $\neg B_{11}$, $\neg S_{11}$, $\neg B_{12}$, $\neg S_{12}$, B_{21}, $\neg S_{21}$. For n = 2: $\neg B_{11}$, $\neg S_{11}$. Each verdict needs two checker runs (α and ¬α), so the models-enumerated figures are 2 × 2^m.

n	Query	Type	Verdict	Symbol set	Models enumerated	Time
2	[1,2] safe	entailed	entailed	full, m = 16	131,072	0.001 s
2	pit at [2,1]	negation entailed	$\neg P_{21}$ entailed	full, m = 16	131,072	< 0.001 s
2	pit at [2,2]	neither	neither	full, m = 16	131,072	< 0.001 s
3	[2,2] safe	entailed	entailed	relevant sentences, m = 18	524,288	0.002 s
3	pit at [1,3]	negation entailed	$\neg P_{13}$ entailed	relevant sentences, m = 18	524,288	0.002 s
3	pit at [3,3]	neither	neither	relevant sentences, m = 19	1,048,576	0.005 s
3	same three queries	–	same	minimal projection, m = 9 to 10	1,024 to 2,048	< 0.0001 s
4	same three queries ([4,4] for "neither")	–	same	relevant sentences, m = 18 to 19	524,288 to 1,048,576	0.002 to 0.005 s
4	same three queries	–	same	minimal projection, m = 9 to 10	1,024 to 2,048	< 0.0001 s

- **Where the full symbol set becomes infeasible.**
  - **n = 2** (2^{16} models) is instant.
  - **n = 3** ($2^{36} \approx 6.9\times10^{10}$ models) is infeasible for a plain one-model-at-a-time checker. I measured about 5×10^5 models/s, so roughly 40 hours. A vectorised checker needs about 13 minutes.
  - **n = 4** (2^{64}) is infeasible for any checker, at about 10^4 years even vectorised.
- **Restriction check.** The "neither" answer from the restricted KB was confirmed against the full KB. I built two complete worlds (one with a pit at [n,n], one without) and checked that both satisfy all 57 or 155 full-KB sentences.

Part B – reasoning patterns

Shared derivations (n = 3). Every step was verified by a truth-table check.

Safety of [2,2] (from $\neg B_{12}$ and $\neg S_{12}$), 11 steps:

1. Biconditional-Elimination on $B_{12} \Leftrightarrow (P_{22} \vee P_{11} \vee P_{13})$.
2. And-Elimination gives $(P_{22} \vee P_{11} \vee P_{13}) \Rightarrow B_{12}$.
3. Modus Ponens / Tollens with $\neg B_{12}$ gives $\neg(P_{22} \vee P_{11} \vee P_{13})$.
4. De Morgan gives $\neg P_{22} \wedge \neg P_{11} \wedge \neg P_{13}$.
5. And-Elimination gives $\mathbf{\neg P_{22}}$.

6. to 10. The same five steps on S_{12} give **$\neg W_{22}$**.
7. And-Introduction gives **$\neg P_{22} \wedge \neg W_{22}$**.

Safety of [1,2] (from $\neg B_{11}$, $\neg S_{11}$) has the same pattern and 11 steps. This gives $\neg P_{12}$ and $\neg W_{12}$.

Bonus: pit at [3,1] (from B_{21}), 5 steps:

1. Biconditional-Elimination on $B_{21} \Leftrightarrow (P_{11} \vee P_{31} \vee P_{22})$.
2. And-Elimination gives $B_{21} \Rightarrow (P_{11} \vee P_{31} \vee P_{22})$.
3. Modus Ponens with B_{21} gives $P_{11} \vee P_{31} \vee P_{22}$.
4. Unit Resolution with $\neg P_{11}$ gives $P_{22} \vee P_{31}$.
5. Unit Resolution with $\neg P_{22}$ gives **P_{31}**.

Student A – step counts against model checking *(measured)*

Fact	Inference steps	Model checking, full KB	Relevant sentences	Minimal projection
safe[1,2]	11	2^{36}	2^{18} = 262,144	2^9
safe[2,2]	11	2^{36}	2^{18}	2^9
pit[3,1]	5	2^{36}	2^{18}	2^9

Rule-based derivation costs about 5 to 11 steps whatever the board size. Model checking costs 2^m. Inference rules win when the proof is short, and the cost is that you must find the proof.

Student B – soundness by truth table. In each case the only rows where all premises are true are listed, and in those rows the conclusion is also true.

Rule	Rows with all premises true	Conclusion in those rows	Verdict
Modus Ponens (α, $\alpha \Rightarrow \beta \vdash \beta$)	α=T, β=T (1 row)	β = T	sound
And-Elimination ($\alpha \wedge \beta \vdash \alpha$)	α=T, β=T (1 row)	α = T	sound
Unit Resolution ($\alpha \vee \beta$, $\neg\alpha \vdash \beta$)	α=F, β=T (1 row)	β = T	sound

The other rules used (biconditional elimination, De Morgan, Modus Tollens, And-Introduction) were checked the same way. Each has no counter-example row.

Why a derived sentence must hold in every model of the KB. Each rule is sound. If a model makes all premises true, it makes the conclusion true. Starting with the KB true in a model, each step keeps the sentences true by induction. So the final sentence is true in every model of the KB.

Why "not derived" is not "false". Inference rules are sound but need not be complete, and our search may simply stop. Failure to derive α shows nothing. α is false ($\neg\alpha$ entailed) only if $\neg\alpha$ is derivable. For [3,3] neither the pit nor its absence is derivable, because both are consistent with the percepts. In logic, "unknown" is a third outcome.

Cross-understanding. Both of us can explain the other's part. Model checking is complete but exponential. Inference rules are fast, sound and need proof search.

TASK 2 – Model Checking versus Resolution

Policy KB (12 rules, includes ⇒, ⇔ and ∨)

Rule	Sentence	CNF clauses
R1	Badge ⇒ Staff ∨ Visitor ∨ Contractor	1
R2	Visitor ⇒ ¬Staff	1
R3	Contractor ⇒ ¬Staff	1
R4	Expired ⇒ ¬Badge	1
R5	Revoked ⇒ ¬Badge	1
R6	InHours ⇔ BizHours ∧ ¬Holiday	3
R7	Override ⇔ FireAlarm ∨ Evacuation	3
R8	Granted ⇔ (Badge ∧ InHours ∧ ¬Lockdown) ∨ Override	5
R9	(Visitor ∧ ¬Escorted ∧ ¬Override) ⇒ ¬Granted	1
R10	Unlocked ⇔ Granted	2
R11	Granted ⇒ Logged	1
R12	ServerRoom ⇔ Granted ∧ Staff ∧ ITDept ∧ ¬Override	5

Facts: Staff, Badge, BizHours, ¬Holiday, ¬Lockdown, ¬FireAlarm, ¬Evacuation. The KB has 19 symbols and 32 clauses.

Student A – resolution investigation

Manual CNF conversion of three sentences.

R6: InHours ⇔ (BizHours ∧ ¬Holiday).

1. Eliminate ⇔: (InHours ⇒ (BizHours ∧ ¬Holiday)) ∧ ((BizHours ∧ ¬Holiday) ⇒ InHours).
2. Eliminate ⇒: (¬InHours ∨ (BizHours ∧ ¬Holiday)) ∧ (¬(BizHours ∧ ¬Holiday) ∨ InHours).
3. De Morgan: second conjunct becomes (¬BizHours ∨ Holiday ∨ InHours).
4. Distribute ∨ over ∧ in the first conjunct.
5. **CNF**: (¬InHours ∨ BizHours) ∧ (¬InHours ∨ ¬Holiday) ∧ (¬BizHours ∨ Holiday ∨ InHours).

R7: Override ⇔ (FireAlarm ∨ Evacuation).

1. After eliminating ⇔ and ⇒: (¬Override ∨ FireAlarm ∨ Evacuation) ∧ (¬(FireAlarm ∨ Evacuation) ∨ Override).
2. De Morgan and distribute.
3. **CNF**: (¬Override ∨ FireAlarm ∨ Evacuation) ∧ (¬FireAlarm ∨ Override) ∧ (¬Evacuation ∨ Override).

R8: Granted ⇔ ((Badge ∧ InHours ∧ ¬Lockdown) ∨ Override). Let X = (Badge ∧ InHours ∧ ¬Lockdown) ∨ Override.

1. (¬Granted ∨ X) ∧ (¬X ∨ Granted).
2. ¬Granted ∨ ((B∧H∧¬L) ∨ O) distributes to (¬G∨B∨O) ∧ (¬G∨H∨O) ∧ (¬G∨¬L∨O).
3. ¬X = (¬B∨¬H∨L) ∧ ¬O, so ¬X ∨ G = (¬B∨¬H∨L∨G) ∧ (¬O∨G).
4. **CNF**, 5 clauses: (¬G∨B∨O) ∧ (¬G∨H∨O) ∧ (¬G∨¬L∨O) ∧ (¬B∨¬H∨L∨G) ∧ (¬O∨G).

My program produced the same clause sets.

Proof of the entailed query "Unlocked". The negated query is ¬Unlocked.

Step	Clause	From	Resolved on
D1	¬BizHours ∨ InHours	(¬BizHours ∨ Holiday ∨ InHours), ¬Holiday	Holiday
D2	InHours	D1, BizHours	BizHours
D3	¬Badge ∨ Granted ∨ Lockdown	(¬Badge ∨ Granted ∨ ¬InHours ∨ Lockdown), D2	InHours
D4	Granted ∨ Lockdown	D3, Badge	Badge
D5	¬Granted	(¬Granted ∨ Unlocked), ¬Unlocked	Unlocked
D6	Lockdown	D4, D5	Granted
D7	□	D6, ¬Lockdown	Lockdown

The proof tree is a chain. D1 → D2 → D3 → D4 and D5 are combined at D6, which is resolved with ¬Lockdown to give D7 = □.

Resolution cost on the policy KB *(measured)*

Query	Strategy	Result	Clauses generated	Resolvents attempted	Tautologies	Subsumed
Unlocked	FIFO	entailed	398	656	96	304
Unlocked	shortest-first	entailed	21	21	0	28
ServerRoom	FIFO	not entailed	876	2,335	98	891
ServerRoom	shortest-first	not entailed	37	42	2	52

Model checking on the same KB enumerates 524,288 models (2^{19}) for either query.

Implication chains (P0, P0⇒P1, …, P(n−1)⇒Pn ⊨ Pn) *(measured)*

n	Model checking	Resolution FIFO (no subsumption), clauses / attempted	Resolution FIFO + subsumption, clauses / attempted / subsumed	Resolution shortest-first + subsumption, clauses / attempted / subsumed
10	2^{11} models, 0.0000 s	66 / 207	66 / 157 / 34	12 / 12 / 10
20	2^{21} models, 0.016 s	231 / 1,251	231 / 912 / 119	22 / 22 / 20
40	$2^{41} = 2.2\times10^{12}$, projected about 4.5 hours	861 / 8,599	861 / 6,176 / 445	42 / 42 / 40
80	$2^{81} = 2.4\times10^{24}$, projected about 6×10^{8} years	3,321 / 63,735	3,321 / 45,354 / 1,721	82 / 82 / 80

- No tautologies occur in this family, so the column is 0 throughout.
- Blind FIFO resolution grows about quadratically (clauses ≈ $n^2/2$). Unit-preferring (shortest-first) selection is linear (n + 2 clauses).
- Subsumption removes many redundant clauses but did not cut the quadratic growth under FIFO. Time was slightly higher with subsumption because of the subsumption checks.

Non-entailed query and termination. ServerRoom is not entailed, because ITDept is unknown. Resolution terminates here because the number of distinct clauses over a finite set of symbols is finite (at most 3^19). Tautologies and duplicates are discarded, so each clause is generated at most once. The loop must eventually produce no new clause, and without □ the answer is "not entailed". Propositional resolution is therefore a decision procedure.

Student B – model-checking comparative analysis *(measured)*

Family	Symbols m	Model checking	Resolution (shortest-first)
Chain, n = 20	21	2,097,152 models	22 clauses
Chain, n = 80	81	infeasible, 2^{81}	82 clauses
Pigeonhole PHP(3,2)	6	64 models	11 clauses
Pigeonhole PHP(4,3)	12	4,096 models, 0.0001 s	244 clauses
Pigeonhole PHP(5,4)	20	1,048,576 models, 0.02 s	16,637 clauses, 0.07 s
Random 3-CNF, 6m clauses, m = 14	14	16,384 models, 0.0004 s	608 clauses
Random 3-CNF, 3m clauses (satisfiable), m = 14	14	16,384 models, 0.0002 s	404 clauses, saturated

- **Family where model checking is faster.** The family is dense, non-Horn, with few symbols: pigeonhole and random 3-CNF with m ≤ 20. Model-checking time depends only on m. The pigeonhole instances need 11, 244 and 16,637 resolution clauses, a growth of about 20 to 70 times per step, and the clause set cannot be shortened. The model-checking time on PHP(5,4) was 0.02 s, against 0.07 s for resolution.
- **Family where model checking is far slower.** The family is a Horn implication chain with many symbols. Resolution needs about n clauses, while 2^(n+1) models are impossible past n ≈ 25.
- **Caveat.** My model checker is vectorised. A one-model-at-a-time checker is roughly 100 times slower, so the "model checking is faster" win applies only to a small m.
- **Structural property.** Model checking depends only on m. Resolution depends on the number of clauses the KB can derive. Horn, sparse and chain-like KBs give short proofs. Dense non-Horn KBs, where many literals can be resolved against many clauses, give an explosive clause closure.

Joint design question

For a reasoning component with unknown KB size and queries, I would want to know:

1. **The number of symbols m.** If m $\lesssim$ 25 and the KB is small, model checking is simple, robust and fast. Beyond that it is impossible.
2. **The structure.** Horn or definite clauses with chains suggest forward chaining or unit resolution. Dense non-Horn clauses with small m suggest model checking or a SAT solver.
3. **The number of queries per KB.** In my measurement of 38 literal queries on the policy KB (m = 19, no facts), re-enumerating per query took 0.71 s. Enumerating the KB's 3,820 models once and caching them took 0.016 s. Resolution, with a fresh proof per query, took 0.137 s. Caching favours model checking when many queries hit a fixed KB.
4. **The proportion of entailed to non-entailed queries.** Non-entailed queries force resolution to saturate, which is costly in non-Horn KBs.
5. **How often the KB changes.** Cached models must be recomputed on a change.

Decision. Use model checking for small m and many queries. Use resolution, or Horn chaining, for large m with sparse structure.

TASK 3 – Knowledge Engineering in First-Order Logic (Exam-Eligibility Advisor)

Knowledge-engineering process

1. **Task.** Decide whether student x may write course c's exam, and explain why not.
2. **Knowledge.** Attendance of at least 75%, or at least 65% with approved leave. All prerequisites passed. Fees paid, or a scholarship. No pending disciplinary action.
3. **Vocabulary** (below).
4. **General knowledge** (axioms A1 to A16).
5. **Instance.** Facts about students, courses, grades and so on.
6. **Queries** (15 below).
7. **Debug.**

Vocabulary.

Kind	Symbols
Constants	students s_1…s_s, courses c_1…c_c, grades
Predicates	Student(x), Course(c), Enrolled(x,c), Prereq(p,c), Grade(x,c,g), PassGrade(g), Passed(x,c), AttPct75(x,c), AttPct65(x,c), ApprovedLeave(x,c), AttOK(x,c), MissingPrereq(x,c), PrereqsOK(x,c), Paid(x), Scholarship(x), FeesOK(x), Disciplinary(x), Cleared(x), Barred(x), Eligible(x,c), HallTicket(x,c), Ineligible(x,c), Reason(x,c,r)

Student A – design derivation

Axioms (16, with quantifiers and one existential).

- A1: ∀x,c [Enrolled(x,c) ∧ AttOK(x,c) ∧ PrereqsOK(x,c) ∧ FeesOK(x) ∧ ¬Barred(x) ⇒ Eligible(x,c)]
- A2: ∀x,c [AttPct75(x,c) ⇒ AttOK(x,c)]
- A3: ∀x,c [AttPct65(x,c) ∧ ApprovedLeave(x,c) ⇒ AttOK(x,c)]
- A4 (existential): ∀x,c [MissingPrereq(x,c) ⇔ ∃p (Prereq(p,c) ∧ ¬Passed(x,p))]
- A5: ∀x,c [¬MissingPrereq(x,c) ⇒ PrereqsOK(x,c)]
- A6: ∀x,c,g [Grade(x,c,g) ∧ PassGrade(g) ⇒ Passed(x,c)]
- A7: ∀x [Paid(x) ⇒ FeesOK(x)]
- A8: ∀x [Scholarship(x) ⇒ FeesOK(x)]
- A9: ∀x [Disciplinary(x) ∧ ¬Cleared(x) ⇒ Barred(x)]
- A10: ∀x,c [Eligible(x,c) ⇒ HallTicket(x,c)]
- A11: ∀x,c [Enrolled(x,c) ∧ ¬Eligible(x,c) ⇒ Ineligible(x,c)]
- A12: ∀x,c [Enrolled(x,c) ∧ ¬AttOK(x,c) ⇒ Reason(x,c,attendance)]
- A13: ∀x,c [Enrolled(x,c) ∧ MissingPrereq(x,c) ⇒ Reason(x,c,prereq)]
- A14: ∀x,c [Enrolled(x,c) ∧ ¬FeesOK(x) ⇒ Reason(x,c,fees)]
- A15: ∀x,c [Enrolled(x,c) ∧ Barred(x) ⇒ Reason(x,c,discipline)]
- A16 (integrity constraint): ∀c [Course(c) ⇒ (Introductory(c) ∨ ∃p Prereq(p,c))]

Negation is read as negation as failure (closed-world assumption) when the axioms are executed.

Growth of the propositional encoding. The propositional encoding needs a copy of each rule per student and course.

Component	Sentences
Per (student, course) pair: eligibility, attendance (2), prereq (about 2), reasons (4), hall ticket, ineligible	about 11 per pair, so ≈ 11·s·c
Per student: fees (2), barred, pass rules	about 4·s
Facts	O(s·c)

N_prop ≈ 11sc + 4s + O(sc) = **Θ(s·c)**.

The FOL encoding has the same 16 axioms whatever s and c are, so N_FOL = **16 + facts = O(1)** in rules. Only the facts grow.

Student B – crossover investigation

15 competency queries (expected results from the KB design; check them against your run).

Five that should succeed:

1. Eligible(s1, c1), with attendance 80%, prerequisites passed, fees paid, no discipline.
2. AttOK(s2, c1), with attendance 68% and approved leave.
3. HallTicket(s1, c1).
4. FeesOK(s3), through a scholarship.
5. Passed(s1, c0).

Five that should fail:

6. Eligible(s4, c1), with attendance 60%.
7. Eligible(s5, c2), with a missing prerequisite.
8. Eligible(s6, c1), with fees unpaid.
9. Eligible(s7, c1), barred (disciplinary and not cleared).
10. AttOK(s8, c1), with attendance 68% and no approved leave.

Five that return bindings:

11. Eligible(x, c1), the list of students.
12. Reason(s4, c1, r), giving r = attendance.
13. Eligible(s1, c), the list of courses.
14. MissingPrereq(s5, c), giving c = c2.
15. Barred(x), the list of barred students.

KB defects to look for and fix (typical defects this KB exposes; log them against your run).

#	Defect	Symptom	Fix
1	A1 lacks ¬Barred(x)	barred student s7 is eligible (query 9 wrongly succeeds)	add ¬Barred(x)
2	A4 polarity: Passed instead of ¬Passed	MissingPrereq(s5,c2) fails, and students with passed prerequisites are blocked	use ∃p (Prereq(p,c) ∧ ¬Passed(x,p))
3	A2 uses AttPct65 instead of AttPct75	s4 (60%) is wrongly judged, and 68% without leave is wrongly judged OK	use AttPct75 in A2

Size and time. These are derived results, not measurements.

- **KB size.** Propositional sentences ≈ 11·s·c. FOL has 16 axioms. FOL is smaller from s·c ≥ 2.

- **Inference time.** After grounding, a propositional query is a dictionary lookup. FOL pays for unification and for searching the subgoals. FOL needs indexing on the first argument to stay fast.
- **Total cost** = build/load + inference. The propositional build grows as s·c, and the FOL build is constant. The FOL encoding becomes preferable as soon as the build/load cost matters, from a few hundred students × a few dozen courses upward, even though each FOL query is slower.

Joint conclusion

Greater expressiveness gives a compact KB (16 axioms against Θ(s·c) sentences), a KB that is easy to maintain and extend, and the ability to ask "which students…" queries. It does not give faster inference in every case. FOL pays for unification, substitutions and search. For a small KB with few students and courses, the propositional encoding answers each query by lookup, so it can be faster. The FOL encoding wins on size, maintenance and build time as s·c grows. Expressiveness gains compactness, not a guarantee of speed.

TASK 4 – Unification and Lifted Inference

Student B – logical design and correctness

Hand-derived unifiers.

#	Pair	Result
1	Knows(John, x) and Knows(John, Jane)	θ = {x/Jane}
2	Knows(John, x) and Knows(y, Mother(y))	y/John, then x/Mother(John). θ = {y/John, x/Mother(John)}
3	Knows(John, x) and Knows(x, Elizabeth)	**Fails** unless standardised apart. Rename the second to Knows(x', Elizabeth). Then θ = {x'/John, x/Elizabeth}. This is why we standardise apart: the two x's are different variables.
4	P(x, y) and P(y, F(x)) (occurs check)	x/y, then y = F(x) = F(y), which gives y/F(y). The occurs check detects y inside F(y) and **fails**. Without it we get an infinite term.
5	Q(x, F(y), G(x,y)) and Q(H(z), F(B), G(H(A), B))	x/H(z); y/B; G(x,y) becomes G(H(z),B) against G(H(A),B), so z/A. After composing, θ = {x/H(A), y/B, z/A}

Why lifted inference is sound. Lifted Modus Ponens takes $p_1' \ldots p_n'$ and $(p_1 \wedge \ldots \wedge p_n \Rightarrow q)$ with $SUBST(\theta, p_i') = SUBST(\theta, p_i)$ and concludes $SUBST(\theta, q)$. The rule is Universal Instantiation followed by ordinary Modus Ponens. Universal Instantiation is sound, and so is Modus Ponens, so the lifted rule is sound.

Why it avoids ground instances. Unification instantiates a rule only as far as the goal needs. With k constants a rule with v variables has k^v ground instances. Lifted inference uses the one rule and derives only the substitutions it needs. The mgu is the most general, so nothing is lost.

Student A – experimental and engineering analysis (exact counts)

Ground clauses against the number of constants. A rule with v distinct variables has k^v ground instances.

k (constants)	1 variable: k	2 variables: k^2	3 variables: k^3
3	3	9	27
5	5	25	125
10	10	100	1,000

k (constants)	1 variable: k	2 variables: k^2	3 variables: k^3
15	15	225	3,375
20	20	400	8,000
25	25	625	15,625
30	30	900	27,000

Arity: a predicate of arity a has k^a ground atoms.

Predicate arity a	Ground atoms k^a (k=10)	Ground instances of a rule using a distinct variables (k=10)
1	10	10
2	100	100
3	1,000	1,000
4	10,000	10,000
5	100,000	100,000

Function symbols. With a function symbol the Herbrand universe is infinite, and propositionalisation needs a depth bound d. With k = 2 constants:

Depth bound d	Terms, unary F (k·(d+1))	Terms, binary G ($T_d = k + T_{\{d-1\}^2}$)
0	2	2
1	4	6
2	6	38
3	8	1,446
4	10	2,090,918

A unary function grows linearly in d. A binary function grows doubly exponentially, so propositionalisation becomes impossible quickly.

Lifted against propositional on the same query. KB: Parent(x,y) ⇒ Anc(x,y); Parent(x,y) ∧ Anc(y,z) ⇒ Anc(x,z); a parent chain over k constants; query Anc(c_0, c_m).

- **Lifted backward chaining** uses about m unifications and m subgoals. This is a derived count. It does not depend on k.
- **Propositional** needs $k^3 + k^2$ ground clauses (k = 30: 27,900) before it starts, then runs on them.

Joint engineering decision

I would propositionalise despite lifted inference when:

1. The domain is **small and fixed** (few constants, no function symbols), so k^v stays small.
2. The KB is **queried very often**, so the grounding cost is amortised.
3. A **fast propositional solver** (SAT or DPLL) is wanted for the full problem, or the KB is non-Horn and needs a complete method.
4. The **rule arity and variable count** are small (v ≤ 2).

I would stay with lifted inference if the number of constants is large, v ≥ 3, function symbols exist, or the KB changes often. The tables above show k^3 growth and unbounded growth with function symbols.

TASK 5 – First-Order Resolution

Student A – conversion and proof design

Step 1: FOL.

1. ∀x∀d [Employee(x) ∧ Restricted(d) ∧ Handled(x,d) ⇒ Authorised(x)]
2. ∀d [Document(d) ∧ PatientData(d) ⇒ Restricted(d)]
3. ∃e [Employee(e) ∧ ∀d (Document(d) ∧ Audited(d) ⇒ Handled(e,d))]
4. Document(R1) ∧ PatientData(R1) ∧ Audited(R1)

Goal: ∃x [Employee(x) ∧ Authorised(x)]. Negated goal: ∀x ¬Employee(x) ∨ ¬Authorised(x).

Step 2: CNF (eliminate ⇒, drop quantifiers after Skolemising).

- C1: ¬Employee(x) ∨ ¬Restricted(d) ∨ ¬Handled(x,d) ∨ Authorised(x)
- C2: ¬Document(d) ∨ ¬PatientData(d) ∨ Restricted(d)
- C3: Employee(E0)
- C4: ¬Document(d) ∨ ¬Audited(d) ∨ Handled(E0,d)
- C5: Document(R1)
- C6: PatientData(R1)
- C7: Audited(R1)
- C8: ¬Employee(x) ∨ ¬Authorised(x) (negated goal)

Skolem constant E0. In statement 3, ∃e is outside every ∀, so it depends on no universal variable and becomes a constant. The universal d is inside the ∃, which is why C4 keeps d and E0 does not depend on it.

Proof (7 resolution steps):

1. C8 + C1 on Authorised(x), (C1 renamed apart, no bindings needed): ¬Employee(x) ∨ ¬Restricted(d) ∨ ¬Handled(x,d). The two ¬Employee(x) literals merge.
2. with C3, {x/E0}: ¬Restricted(d) ∨ ¬Handled(E0,d).
3. with C2: ¬Document(d) ∨ ¬PatientData(d) ∨ ¬Handled(E0,d).
4. with C4: ¬Document(d) ∨ ¬PatientData(d) ∨ ¬Audited(d). The two ¬Document(d) literals merge.
5. with C5, {d/R1}: ¬PatientData(R1) ∨ ¬Audited(R1).
6. with C6: ¬Audited(R1).
7. with C7: □.

So an authorised employee exists: **E0**.

Strategy: set of support. Every resolution uses the negated goal C8, or a descendant of it, as one parent. This is exactly the chain above.

Student B – comparative evidence

The proof above is **7 resolution steps** and uses 8 input clauses. Unrestricted breadth-first resolution would also resolve axiom pairs that do not involve the goal, such as C2 + C5 → ¬PatientData(R1) ∨ Restricted(R1) (this one is useful). It would also generate clauses unrelated to the goal. Set of support avoids all resolutions between input axioms.

Non-entailed query. Take “some authorised contractor exists”: ∃x [Authorised(x) ∧ Contractor(x)].

- **On this KB alone**, the search terminates. There is a finite Herbrand universe {E0, R1}, so the set of clauses is finite. It saturates without finding □.
- **If an axiom such as "every employee has a boss who is an employee" is added**, Skolemised to ¬Employee(x) ∨ Employee(boss(x)), the terms boss(E0), boss(boss(E0)), … are generated forever. The prover never terminates by itself. A **step cap** stops it and reports "unknown".

Compared with the propositional prover of Task 2. On a domain of k constants the FOL KB grounds to about $k^2 + 2k$ clause instances for the rules. The FOL proof stays at 7 steps whatever the domain size. The propositional proof needs the grounded domain first.

Cross-understanding answers

Student B: why Skolemisation preserves satisfiability but not logical equivalence.

- Replacing ∃e ϕ(e) by ϕ(E0) gives a sentence with a new symbol E0. If the original KB has a model, we can interpret E0 as the witness, so the new KB is satisfiable. If the new KB has a model, the witness E0 shows the original is satisfiable. So **satisfiability is preserved**, and refutation is valid.
- They are not equivalent. The new sentence mentions E0, which the original does not, and ϕ(E0) is a stronger claim: it says a specific named object satisfies ϕ. The two sentences can differ in other models.

Student A: what the evidence shows about the cost of lifting.

- Lifting adds the cost of unification, standardising apart and the substitution bookkeeping to every step. In return the proof is 7 steps against many for grounding, and it does not depend on the domain size.
- **What the non-entailed query shows.** First-order entailment is **semi-decidable**. If KB ⊨ α, resolution eventually finds □. If KB ⊭ α, the procedure may run forever when function symbols exist, as with boss(·). A step cap is a practical answer, but cannot tell "not entailed" from "not yet found".

TASK 6 – Forward and Backward Chaining

KB (organisation hierarchy, 10 definite clauses).

1. ChainOfCommand(e,m) ⇐ ReportsTo(e,m)
2. ChainOfCommand(e,m) ⇐ ReportsTo(e,x) ∧ ChainOfCommand(x,m)
3. Subordinate(m,e) ⇐ ChainOfCommand(e,m)
4. DirectManager(m,e) ⇐ ReportsTo(e,m)
5. CanApprove(m,r) ⇐ Request(r,e) ∧ DirectManager(m,e) ∧ Small(r)
6. CanApprove(m,r) ⇐ Request(r,e) ∧ Subordinate(m,e) ∧ Director(m) ∧ Medium(r)
7. CanApprove(m,r) ⇐ Request(r,e) ∧ Subordinate(m,e) ∧ Executive(m) ∧ Large(r)
8. PeerOf(a,b) ⇐ ReportsTo(a,m) ∧ ReportsTo(b,m)
9. ConflictOfInterest(a,r) ⇐ Request(r,e) ∧ PeerOf(a,e)
10. Authorised(m,r) ⇐ CanApprove(m,r) ∧ HasBudgetAuthority(m)

Student A – forward chaining

Engine. Forward chaining to a fixed point. In each iteration it applies every rule whose premises match the known facts, adds new conclusions, and stops when an iteration adds nothing.

What I expect for the metrics (derived from the structure, not measured). For a reporting tree with n employees and depth about $\log_4 n$:

- **Facts inferred** grow roughly as n × depth for ChainOfCommand and Subordinate. PeerOf and ConflictOfInterest add about n × b more (b = number of siblings).
- **Iterations to fixed point** ≈ tree depth + a few.
- **Redundant derivations** are significant: PeerOf(a,b) and PeerOf(b,a) arise independently, and a ChainOfCommand fact can be derived from several paths.
- **Facts irrelevant to the query** are the large majority. For one query such as Authorised(m, r) only one request, one chain of managers and a few attribute facts matter. With 10,000 facts, I expect well over 90% of derived facts to be irrelevant.

When the query concerns only a small part of the KB. Forward chaining does **all the work regardless**, because it is data-driven and has no knowledge of the query. The work scales with the whole KB, so the cost is paid for facts that cannot contribute to the answer.

Student B – backward chaining

Engine. Backward chaining with substitutions: take the goal, find rules whose head unifies with it, standardise apart, and recursively prove the body goals with the substitution carried along. Facts are indexed on predicate and first argument.

Non-termination. Replace rule 2 with the left-recursive rule

ChainOfCommand(e,m) ⇐ ChainOfCommand(e,x) ∧ ReportsTo(x,m)

The first subgoal ChainOfCommand(e,x) calls the same rule again before anything is bound, so the depth grows without bound and naive backward chaining never terminates.

Remedies.

Remedy	Effect	Cost / limits
Depth bound (iterative deepening)	terminates, complete if the bound ≥ tree depth	repeats work; the bound must be chosen
Loop check (ancestor variant check)	terminates	**incomplete**: prunes answers needing recursion
Memoisation / tabling (store subgoal answers, re-iterate to a fixed point)	terminates and complete for Datalog	memory for the table; extra bookkeeping

I would choose tabling for correctness.

Metrics to record (what the engine reports): goals expanded, unifications, maximum depth, repeated subgoals. For the right-recursive rule the depth equals the depth of the tree, so it grows as log n. The repeated subgoals are the ones tabling removes.

Verification questions

Student B: why is the choice of inference direction central to performance on this KB? The KB has many facts (100 to 10,000), but each query concerns one request and one chain of managers. Forward chaining derives everything and so costs time proportional to the whole KB. Backward chaining starts from the goal and touches only the relevant facts. Direction decides whether the cost is proportional to the KB or to the proof.

Student A: why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query? Backward chaining is goal-directed. It only looks for facts that unify with a subgoal, and with first-argument indexing each lookup is a hash access. Irrelevant facts are never touched. The cost depends on the proof's size and branching, not on the total number of facts.

TASK 7 – Design Challenge: Hospital Medication-Safety Advisor

The clinical rules below are illustrative, to demonstrate the design. They are not medical advice.

Stage 1 – independent design proposals

Student A's initial proposal.

- Representation: propositional logic. One symbol per patient/drug fact, e.g. Allergic_p17_penicillin.
- Vocabulary: symbols for allergies, drugs, conditions, age bands.
- Inference: forward chaining over Horn rules.
- Explanations: list the rules that fired.

Student B's initial proposal.

- Representation: first-order logic with variables for patients and drugs.
- Vocabulary: Takes(p,d), Allergic(p,a), Age(p,n), HasCondition(p,c), Prescribed(p,d), Unsafe(p,d,reason).
- Inference: backward chaining from the query Unsafe(p,d,r).
- Explanations: the proof tree.

Stage 2 – comparison

	Agree	Differ
Common	Horn-style rules; explanations from the derivation	–
A	–	Propositional: needs one copy of every rule per patient, so 50,000 patients × 2,000 rules is too big
A	–	Forward chaining derives all consequences for all patients
B	–	FOL: 2,000 general rules stay constant as patients grow
B	–	Backward chaining answers only the new prescription

Strengths: A is simple and fast per lookup. B scales with patients and gives natural proofs. Weaknesses: A explodes with patients, and B needs indexing to avoid scanning all rules. **Retained:** FOL from B, Horn clauses from both, derivation-based explanations from both, and forward chaining from A for patient-independent knowledge such as drug-class closure.

Stage 3 – joint final design

- **Representation:** function-free FOL (Datalog) with definite clauses and negation as failure for "no reason found".
- **Vocabulary:** Patient facts: Age(p,n), Allergic(p,a), Takes(p,d), HasCondition(p,c), Prescribed(p,d). Drug knowledge: In(d,class), SubclassOf, MinAge(d,n), Interacts($class_1$,$class_2$,severity), ContraCondition(d,c). Output: Unsafe(p,d,reason).
- **Rules** (examples):

  - Unsafe(p,d,allergy) ⇐ Prescribed(p,d) ∧ Allergic(p,a) ∧ In(d,a)
  - Unsafe(p,d,interaction(m)) ⇐ Prescribed(p,d) ∧ Takes(p,m) ∧ In(d,k_1) ∧ In(m,k_2) ∧ Interacts(k_1,k_2,s)
  - Unsafe(p,d,age) ⇐ Prescribed(p,d) ∧ Age(p,a) ∧ MinAge(d,n) ∧ a < n
  - Unsafe(p,d,condition(c)) ⇐ Prescribed(p,d) ∧ HasCondition(p,c) ∧ ContraCondition(d,c)
  - In(d,k_2) ⇐ In(d,k_1) ∧ SubclassOf(k_1,k_2)
- **Inference method:** backward chaining from Unsafe(p,d,R), with tabling for the recursive class rule.
- **Indexing and agenda for scale:**
  1. Index rules by head predicate and by the drug constant in the head, so a query for one drug sees a handful of the 2,000 rules.
  2. Index patient facts by patient id, so only that patient's facts are touched among 50,000 patients.
  3. Precompute the drug-class closure In(d,k) once by forward chaining, since it does not depend on the patient.
  4. Order subgoals so that the most selective (bound) goal is tried first.
- **Explanation mechanism:** every successful derivation is kept as a proof tree. The output reads "Unsafe because Prescribed(p,warfarin) and Takes(p,aspirin) and Interacts(anticoagulant, NSAID, high)".

Pseudocode.

```
function CHECK(patient p, drug d):
    reasons = []
    for each answer in BC-ASK(Unsafe(p, d, R)):        # indexed, tabled
        reasons.append((R, PROOF-TREE(answer)))
    if reasons is empty:
        return "NO KNOWN CONTRAINDICATION", caveat "closed-world: record may be incomplete"
    return "UNSAFE", reasons

function BC-ASK(goal):
    for each rule in RULES-INDEXED-BY(goal.predicate, goal.constant args):
        rule' = STANDARDISE-APART(rule)
        theta = UNIFY(goal, head(rule'))
        if theta fails: continue
        for each sigma in PROVE-ALL(SUBST(theta, body(rule'))):
            yield COMPOSE(theta, sigma)
    for each fact in FACTS-INDEXED-BY(goal):           # patient id is the key
        theta = UNIFY(goal, fact);  if theta: yield theta
```

Stage 4 – validation

Student A – theoretical validation.

- **Soundness.** Each derivation uses Modus Ponens on ground instances of rules (via unification). Each step is sound, so any Unsafe verdict follows logically from the KB.
- **Completeness.** Backward chaining with tabling is complete for Datalog Horn clauses. Without tabling it can loop on recursive rules. For plain definite clauses without negation, every entailed Unsafe fact is found.
- **Complexity of a query.** Data complexity of Datalog is polynomial. For a goal with an indexed rule set of size r_d (rules for that drug) and b facts for that patient, the cost is about O(r_d × b^v) with v body variables. In particular it does not grow with the total number of rules (2,000) or patients (50,000)

when the indexes are used. Without indexes the cost is O(|rules|) per goal. With propositional forward chaining, a full run is O(|KB|), linear in the KB.

- **Effect of restricting to definite clauses.** Inference is polynomial instead of NP-hard (SAT) or co-NP-complete (entailment). The cost is expressiveness: no disjunctive knowledge (for example "allergic to A or B, unknown which"), and negation only as failure (a closed-world assumption).

Student B – experimental validation (test design). I have not run these tests. The designed test suite (30 or more cases) is:

Category	Cases	Example	Expected verdict
Dangerous combinations	10	warfarin + aspirin	Unsafe (bleeding)
		ACE inhibitor + potassium-sparing diuretic	Unsafe
		SSRI + MAO inhibitor	Unsafe
		nitrate + sildenafil	Unsafe
Allergy	5	penicillin allergy, amoxicillin prescribed	Unsafe
Age	5	child under the drug's minimum age	Unsafe
Condition	5	NSAID with peptic ulcer	Unsafe
Safe	5	no matching rule	No known contraindication
Edge	3	patient with no record; two reasons at once; allergy class via subclass	handled

Run at **KB sizes 40, 400 and 2,000 rules** and record query latency, inference steps (unifications), memory and explanation length, against two baselines: forward chaining over all patients, and backward chaining without indexes. The same 30 cases must give the same verdicts at every size. Expected pattern: with indexing, latency and steps stay almost flat as the KB grows from 40 to 2,000 rules. The unindexed baseline grows roughly linearly with the rule count.

Stage 5 – cross-defence

Student A (experimental evidence supporting the design). The 30-case suite must agree at all three sizes. Indexed latency must stay flat while the unindexed baseline grows. That is evidence that the design's cost depends on the relevant rules and not on KB size.

Student B (why the complexity result applies). The final system is Datalog definite clauses with tabled backward chaining. Datalog entailment has polynomial data complexity, and indexing makes the cost depend on the matching rules and the one patient's facts. That is why 2,000 rules and 50,000 patients do not make a query's cost grow.

Stage 6 – joint evaluation

Why it suits a logical agent. The knowledge is explicit, rule-like and stable (contraindications, interaction tables). Decisions must be explained to a clinician, and a proof gives that. Rules can be reviewed, audited and updated one at a time. Correctness (soundness) matters more than statistical performance.

Limits.

- **Uncertainty.** Logic gives yes or no. It cannot say "interaction probable" or weigh risk against benefit.

- **Incomplete data.** Negation as failure treats a missing allergy record as "no allergy". That is dangerous. The system must say "no known contraindication", never "safe".
- **Brittleness.** Unmodelled drugs and rare cases are silently missed. Exceptions and dose-dependence (renal function, dose size) need extra rules.
- **Maintenance.** 2,000 rules need curation and conflict checking.

What a designer should check before choosing a knowledge-based approach.

1. Is the knowledge explicit, stable and expressible as rules?
2. Is an explanation required?
3. How complete and reliable is the input data?
4. Is the uncertainty small enough to ignore, or does a probabilistic or learned model need to be combined?
5. Can the knowledge be maintained as it grows?
6. What are the cost and consequences of a wrong answer?

CONTRIBUTION RECORD

Edit this to reflect the work each of you actually did.

Task	Student A – primary contribution	Student B – primary contribution	Joint work
Task 1	Symbol/sentence counts, model count, minimal symbol set; inference-rule derivations and step counts	Truth-table checker experiments (n = 2, 3, 4); soundness proofs	KB encoding, query design
Task 2	Manual CNF, resolution proof, clause metrics on chains, non-entailed query	Model-checking measurements; families where each method wins	Policy KB (12 rules), joint design discussion
Task 3	FOL vocabulary and axioms, growth derivation	15 queries, defect log, size/time analysis	Two encodings, conclusion
Task 4	Ground-clause growth, function-symbol growth, lifted vs propositional	Hand-derived unifiers, soundness of lifted inference	UNIFY, lifted MP, propositionalisation, engineering decision
Task 5	CNF conversion, Skolem constant, proof tree, strategy	Strategy comparison, non-entailed query, Task 2 comparison	Prover design, cross-understanding answers
Task 6	Forward chaining and its answers	Backward chaining, left-recursion remedy and its answers	KB of 10 rules
Task 7	Design A, theoretical validation, cross-defence	Design B, experimental validation, cross-defence	Comparison, final design, joint evaluation